Módulo extra 3 - Firebase

Sumário

1. Introdução ao Firebase – O Que É e Por Que Usá-lo no Desenvolvimento Moderno

Seja bem-vindo a este novo módulo extra! Depois de explorarmos o desenvolvimento backend com Node.js e a construção de interfaces reativas com React, vamos dar um salto em direção ao ecossistema de BaaS (Backend-as-a-Service) conhecendo o Firebase.

Neste primeiro capítulo, vamos entender o conceito por trás do Firebase, quais problemas ele resolve e por que ele se tornou uma das ferramentas mais populares para colocar aplicações no ar com rapidez e alta escalabilidade.

1.1. O Problema do Desenvolvimento Tradicional

Até agora, vimos que para criar uma aplicação completa precisamos de:

- Uma máquina ou container na nuvem para hospedar o servidor backend (Node.js).
- A configuração de um banco de dados relacional ou não-relacional.
- A implementação manual de sistemas complexos de autenticação de usuários (criptografia de senhas, tokens JWT, recuperação de senha).
- A configuração de servidores de arquivos para armazenar imagens e documentos enviados pelos usuários.

Embora esse modelo tradicional ofereça controle total, ele exige muito tempo de infraestrutura antes mesmo de o produto principal começar a ser construído.

1.2. O Que é o Firebase?

O Firebase é uma plataforma de desenvolvimento de aplicativos móveis e web comprada e mantida pelo Google. Ele se enquadra na categoria de BaaS (Backend-as-a-Service).

Em vez de você precisar construir toda a infraestrutura do backend do zero, o Firebase fornece um backend pronto e gerenciado na nuvem, permitindo que você conecte o seu frontend (como um aplicativo React ou mobile) diretamente aos serviços essenciais da aplicação através de SDKs oficiais.

Os Pilares do Ecossistema Firebase:

- Firebase Authentication: Sistema pronto de login e cadastro (por e-mail/senha, Google, GitHub, telefone, etc.) sem que você precise programar regras de criptografia ou segurança de senhas.

- Cloud Firestore / Realtime Database: Bancos de dados NoSQL em nuvem que armazenam dados em tempo real, sincronizando alterações instantaneamente entre os clientes conectados.

- Firebase Hosting: Serviço de hospedagem ultrarrápido para aplicações web estáticas e SPAs (Single Page Applications).

- Cloud Storage: Armazenamento seguro de arquivos binários (como fotos de perfil, vídeos e PDFs) diretamente na nuvem do Google.

- Cloud Functions: Execução de códigos backend em JavaScript/TypeScript acionados por eventos do Firebase, rodando em servidores sem que você precise gerenciar infraestrutura (serverless).

1.3. Vantagens e Desvantagens do Modelo BaaS

Como qualquer tecnologia na engenharia de software, o Firebase traz trade-offs (escolhas com vantagens e custos) que devemos considerar:

Vantagens:

- Velocidade de Desenvolvimento (Time-to-Market): Você consegue lançar um MVP (Mínimo Produto Viável) em dias, em vez de meses.

- Escalabilidade Automática: O Google gerencia toda a infraestrutura de servidores, banco de dados e tráfego de rede por você.

- Sincronização em Tempo Real: Alterou um dado no banco, a interface do usuário é atualizada instantaneamente sem recarregar a página.

Desvantagens e Cuidados:

- Vendor Lock-in (Acoplamento com o Provedor): Migrar de ecossistema depois que a aplicação está totalmente presa aos serviços proprietários do Google pode ser custoso e trabalhoso.

- Custos em Escala: Para projetos pequenos ou em fase de testes, o plano gratuito (Spark) é generoso. No entanto, à medida que a aplicação cresce e o número de leituras/escritas no banco dispara, os custos podem se elevar rapidamente se a arquitetura das consultas não for otimizada.

2. Configuração Inicial – Criando o Projeto no Console e Instalando o SDK

No Capítulo 1, entendemos o conceito de BaaS (Backend-as-a-Service) e conhecemos os principais pilares do Firebase, a plataforma do Google que nos poupa o trabalho de construir servidores e bancos de dados tradicionais do zero.

Agora que a teoria está clara, chegou o momento de colocar a mão na massa. Neste capítulo, vamos criar o nosso primeiro projeto no painel do Firebase, registrar uma aplicação web e instalar o SDK oficial no nosso ambiente de desenvolvimento.

2.1. Criando o Projeto no Console do Firebase

- O primeiro passo é configurar a nossa "casa" na nuvem do Google:

- Acesse o site oficial do console do Firebase: console.firebase.google.com e faça login com a sua conta Google.

- Clique no botão "Criar projeto" (ou "Adicionar projeto").

- Nome do Projeto: Digite um nome para a sua aplicação (por exemplo, meu-app-firebase).

- Google Analytics: O Firebase perguntará se você deseja ativar o Google Analytics para o projeto. Para fins de estudo e desenvolvimento inicial, você pode desativar essa opção (ou mantê-la ativada, caso queira métricas de uso).

- Clique em "Criar projeto" e aguarde alguns segundos enquanto o Google provisiona os recursos na nuvem. Quando estiver pronto, clique em "Continuar".

2.2. Registrando a Aplicação Web e Obtendo as Credenciais

Dentro do painel do seu novo projeto, você precisa informar ao Firebase onde essas ferramentas serão utilizadas (seja em um app Android, iOS ou Web). Como estamos focados em aplicações web (como as que construímos com React ou Node.js):

- Na página inicial do projeto, clique no ícone da Web (</>) para registrar um aplicativo web.

- Apelido do app: Dê um nome amigável para a sua aplicação (ex: frontend-web).

- Marque ou desmarque a opção de configurar o Firebase Hosting (por enquanto, podemos deixar desmarcado) e clique em "Registrar app".

- O Firebase exibirá um trecho de código contendo o firebaseConfig (um objeto JavaScript com chaves como apiKey, authDomain, projectId, etc.).

Aviso de Segurança: Embora as chaves do Firebase do lado do cliente fiquem visíveis no navegador do usuário, você deve protegê-las utilizando variáveis de ambiente (.env) em projetos reais para evitar o uso indevido das suas cotas de recursos.

2.3. Instalando o Firebase no Projeto via NPM

Com as credenciais em mãos, vamos voltar ao nosso terminal e instalar o pacote oficial do Firebase na pasta do nosso projeto (assumindo que você já tenha inicializado o projeto com npm init -y ou esteja usando uma ferramenta como Vite/React):

```
npm install firebase
```

O que aconteceu aqui?
O NPM baixou o SDK modular do Firebase para a pasta node_modules e o registrou no package.json. Esse pacote é a biblioteca oficial que permite ao nosso código JavaScript conversar diretamente com a nuvem do Google (para autenticar usuários, salvar dados no banco ou enviar arquivos).

2.4. Inicializando o Firebase no Código (firebase.js)

Para manter o código limpo e organizado, criamos um arquivo dedicado (por exemplo, firebase.js) na pasta do projeto para centralizar a conexão:

```
// firebase.js
import { initializeApp } from "firebase/app";

// As credenciais fornecidas pelo console do Firebase
const firebaseConfig = {
```

```
  apiKey: "SUA_API_KEY_AQUI",
  authDomain: "seu-projeto.firebaseapp.com",
  projectId: "seu-projeto",
  storageBucket: "seu-projeto.appspot.com",
  messagingSenderId: "SEU_SENDER_ID",
  appId: "SEU_APP_ID"
};

// Inicializando o Firebase na nossa aplicação
const app = initializeApp(firebaseConfig);

export { app };
```

Ao importar e exportar essa inicialização, qualquer outro arquivo do nosso sistema poderá solicitar os serviços do Firebase (como Autenticação ou Banco de Dados) de forma imediata.

3. Firebase Authentication – Implementando Login e Cadastro de Forma Segura

No Capítulo 1, conhecemos o conceito de BaaS e a proposta do Firebase. No Capítulo 2, criamos o nosso projeto no console, obtivemos as credenciais e instalamos o SDK oficial via NPM (npm install firebase).

Agora que o terreno está pronto, vamos explorar um dos serviços mais poderosos e utilizados da plataforma: o Firebase Authentication.

3.1. O Problema da Autenticação Tradicional

Construir um sistema de login e cadastro do zero no backend tradicional exige uma quantidade enorme de trabalho repetitivo e sensível:

- Criar tabelas de usuários no banco de dados.
- Implementar algoritmos complexos de criptografia e hashing de senhas (como bcrypt ou Argon2) para nunca salvar senhas em texto puro.
- Gerenciar tokens de sessão (JWT), cookies seguros, fluxos de recuperação de senha ("Esqueci minha senha") e segurança contra ataques de força bruta.

- Integrar provedores externos (como login com Google, GitHub ou Apple).

O Firebase Authentication resolve tudo isso em poucas linhas de código, gerenciando a infraestrutura de segurança e a criptografia nos servidores do Google.

3.2. Ativando o Authentication no Console do Firebase

Antes de escrever código, precisamos habilitar o provedor de login que desejamos usar no painel do Google:

- Acesse o Console do Firebase e abra o seu projeto.

- No menu lateral esquerdo, clique em Build (Desenvolver) e selecione Authentication.

- Clique no botão "Começar" (Get Started).

- Na aba Sign-in method (Método de login), escolha a opção E-mail/Senha (Email/Password).

Ative a chave de Habilitar (Enable) e clique em Salvar. (Opcionalmente, você pode ativar o link de e-mail se desejar, mas o padrão é senha).

3.3. Escrevendo o Código de Autenticação

Com o serviço ativo no console, vamos criar um arquivo de serviço (por exemplo, auth.js) para centralizar as funções de cadastro e login, utilizando o SDK modular do Firebase.

A. Configurando os Módulos de Auth

```
// auth.js
import { getAuth, createUserWithEmailAndPassword,
signInWithEmailAndPassword, signOut } from "firebase/auth";
import { app } from "./firebase"; // Importamos a instância do app
que criamos no capítulo anterior

// Inicializamos o serviço de autenticação associado ao nosso app
const auth = getAuth(app);

// 1. Função para Cadastrar um Novo Usuário
```

```
export async function cadastrarUsuario(email, senha) {
    try {
        const userCredential = await
createUserWithEmailAndPassword(auth, email, senha);
        const usuario = userCredential.user;

        console.log("Usuário cadastrado com sucesso!",
usuario.uid);
        return usuario;
    } catch (error) {
        console.error("Erro ao cadastrar:", error.message);
        throw error;
    }
}

// 2. Função para Fazer Login de um Usuário Existente
export async function fazerLogin(email, senha) {
    try {
        const userCredential = await
signInWithEmailAndPassword(auth, email, senha);
        const usuario = userCredential.user;

        console.log("Login realizado com sucesso!",
usuario.email);
        return usuario;
    } catch (error) {
        console.error("Erro ao fazer login:", error.message);
        throw error;
    }
}

// 3. Função para Fazer Logout (Sair da Conta)
export async function fazerLogout() {
    try {
        await signOut(auth);
        console.log("Usuário desconectado com sucesso.");
    } catch (error) {
        console.error(" Erro ao sair:", error.message);
    }
}
```

3.4. Como Integrar com o Frontend (Exemplo com React)

Em uma aplicação real (como as que construímos com React nos módulos anteriores), costumamos amarrar essas funções aos eventos de clique de um formulário de login/cadastro ou a botões de ação:

```
import { useState } from 'react';
import { cadastrarUsuario, fazerLogin } from './auth';

function TelaAutenticacao() {
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  async function handleCadastro(e) {
    e.preventDefault();
    try {
      await cadastrarUsuario(email, senha);
      alert('Conta criada com sucesso!');
    } catch (err) {
      alert('Falha ao cadastrar. Verifique os dados.');
    }
  }

  async function handleLogin(e) {
    e.preventDefault();
    try {
      await fazerLogin(email, senha);
      alert('Bem-vindo de volta!');
    } catch (err) {
      alert('E-mail ou senha inválidos.');
    }
  }

  return (
    <div style={{ padding: '20px', maxWidth: '300px' }}>
      <h2>Acesso ao Sistema</h2>
      <form>
        <input
          type="email"
          placeholder="E-mail"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          style={{ width: '100%', marginBottom: '10px', padding:
'8px' }}
```

```
        />
        <br />
        <input
          type="password"
          placeholder="Senha"
          value={senha}
          onChange={(e) => setSenha(e.target.value)}
          style={{ width: '100%', marginBottom: '10px', padding:
'8px' }}
        />
        <br />
        <button onClick={handleLogin} style={{ marginRight:
'10px', padding: '8px' }}>Entrar</button>
        <button onClick={handleCadastro} style={{ padding: '8px'
}}>Cadastrar</button>
      </form>
    </div>
  );
}

export default TelaAutenticacao;
```

4. Cloud Firestore – Salvando e Lendo Dados na Nuvem em Tempo Real

Nos capítulos anteriores, instalamos o Firebase, configuramos o SDK e implementamos um sistema completo de autenticação de usuários sem precisar construir tabelas ou fluxos de criptografia no backend.

Agora, precisamos de um lugar para guardar as informações da nossa aplicação (como tarefas, posts, perfis de usuários ou mensagens). Vamos explorar o Cloud Firestore, o banco de dados NoSQL em nuvem do Firebase.

4.1. O Que é o Cloud Firestore?

Diferente dos bancos de dados relacionais tradicionais (como o MySQL que vimos nos módulos anteriores, organizados em tabelas, linhas e colunas fixas), o Cloud Firestore é um banco de dados NoSQL orientado a documentos.

Como ele organiza os dados?

- Coleções (Collections): Funcionam como pastas que agrupam documentos semelhantes (ex: uma coleção chamada usuarios ou tarefas).

- Documentos (Documents): Funcionam como arquivos individuais dentro dessas coleções. Cada documento é um objeto em formato JSON (composto por pares de chave-valor) que possui um ID único gerado automaticamente pelo sistema.

4.2. Ativando o Firestore no Console

Antes de gravar dados pelo código, precisamos provisionar o banco de dados na nuvem:

- No Console do Firebase, acesse o seu projeto.

- No menu lateral esquerdo, clique em Build e selecione Firestore Database.

- Clique em Criar banco de dados (Create Database).

- Localização do Servidor: Escolha a região mais próxima dos seus usuários (ex: nam5 para as Américas ou a região recomendada pelo painel) para reduzir a latência de rede.

- Modo de Segurança: Para fins de testes iniciais, escolha o Modo de Teste (Test Mode). Isso permite leituras e gravações públicas temporárias sem regras complexas de segurança (lembre-se de configurar regras restritivas antes de ir para produção!).

- Clique em Ativar.

4.3. Escrevendo e Lendo Dados na Prática

Com o Firestore ativo, vamos criar um arquivo de serviço (por exemplo, firestore.js) utilizando as funções do SDK modular do Firebase para realizar as operações fundamentais de salvamento e consulta.

A. Inicializando o Firestore

```
// firestore.js
import { getFirestore, collection, addDoc, getDocs, doc,
updateDoc, deleteDoc } from "firebase/firestore";
import { app } from "./firebase";

// Inicializa a instância do banco de dados
const db = getFirestore(app);
```

```
// 1. Criando (Salvando) um novo documento em uma coleção
export async function adicionarTarefa(titulo, prioridade) {
    try {
        // O addDoc cria automaticamente um documento com ID único
na coleção "tarefas"
        const docRef = await addDoc(collection(db, "tarefas"), {
            titulo: titulo,
            prioridade: prioridade,
            concluida: false,
            criadoEm: new Date()
        });

        console.log("Tarefa salva com o ID: ", docRef.id);
        return docRef.id;
    } catch (e) {
        console.error("Erro ao adicionar documento: ", e);
        throw e;
    }
}

// 2. Lendo (Buscando) todos os documentos de uma coleção
export async function listarTarefas() {
    try {
        const querySnapshot = await getDocs(collection(db,
"tarefas"));
        const tarefas = [];

        querySnapshot.forEach((doc) => {
            // Em cada documento, extraímos os dados e adicionamos
o ID gerado pelo Firebase
            tarefas.push({
                id: doc.id,
                ...doc.data()
            });
        });

        console.log("Tarefas recuperadas:", tarefas);
        return tarefas;
    } catch (e) {
        console.error("Erro ao buscar documentos: ", e);
        throw e;
```

```
    }
}
```

4.4. O Poder do Tempo Real (Realtime Listeners)

Uma das características mais marcantes do Cloud Firestore é a capacidade de escutar alterações em tempo real através da função onSnapshot.

Enquanto com getDocs fazemos uma requisição pontual (como dar um "Refresh" na página), o ouvinte em tempo real avisa o seu componente React instantaneamente sempre que qualquer usuário alterar, adicionar ou remover um dado no banco, sem precisar recarregar nada.

```
import { collection, onSnapshot } from "firebase/firestore";

// Exemplo de escuta em tempo real para atualizar o estado de um componente
export function escutarTarefasEmTempoReal(callbackAtualizacao) {
    const colRef = collection(db, "tarefas");

    // O onSnapshot fica "ouvindo" o banco continuamente
    onSnapshot(colRef, (querySnapshot) => {
        const tarefas = [];
        querySnapshot.forEach((doc) => {
            tarefas.push({ id: doc.id, ...doc.data() });
        });

        // Executa a função de callback repassando a lista atualizada para o componente
        callbackAtualizacao(tarefas);
    });
}
```

5. Cloud Storage – Armazenando e Gerenciando Arquivos na Nuvem

Nos capítulos anteriores, instalamos o Firebase, configuramos o SDK, implementamos o sistema de autenticação de usuários e aprendemos a salvar e ler dados em tempo real no Cloud Firestore.

No entanto, sistemas reais raramente lidam apenas com textos e números. O que acontece quando o usuário precisa enviar uma foto de perfil, anexar um documento PDF, fazer upload de imagens para um produto ou enviar um arquivo de áudio? É exatamente para isso que serve o Cloud Storage.

5.1. O Que é o Cloud Storage?

O Cloud Storage é um serviço de armazenamento de objetos altamente escalável e seguro, projetado para guardar arquivos binários pesados (como imagens, vídeos, áudios e documentos).

- Diferença para o Firestore: Enquanto o Cloud Firestore guarda dados estruturados em texto e JSON (nomes, preços, e-mails, status), o Cloud Storage guarda arquivos brutos e gera uma URL pública de acesso (ou URL de download) para que a sua aplicação possa exibir ou baixar esse arquivo quando necessário.

- Segurança e Regras: Os arquivos são salvos em um "Bucket" na nuvem do Google e podem ser protegidos por regras rígidas de acesso (permitindo que apenas usuários autenticados façam upload, por exemplo).

5.2. Ativando o Storage no Console do Firebase

Antes de enviar qualquer arquivo pelo código, precisamos provisionar o serviço no painel:

- No Console do Firebase, acesse o seu projeto.

- No menu lateral esquerdo, clique em Build e selecione Storage.

- Clique em Começar (Get Started).

- Escolha o Modo de Teste (Test Mode) para permitir gravações iniciais sem regras restritivas de segurança bloqueando os testes.

- Selecione a região do servidor e clique em Concluído.

5.3. Realizando o Upload de Arquivos na Prática

Para interagir com o Storage via código, utilizamos as funções do SDK modular do Firebase. Vamos criar um arquivo de serviço (por exemplo, storage.js) contendo a lógica para enviar um arquivo selecionado pelo usuário e obter o link de acesso direto.

Escrevendo a função de Upload:

```
// storage.js
```

```
import { getStorage, ref, uploadBytes, getDownloadURL } from
"firebase/storage";
import { app } from "./firebase";

// Inicializa a instância do Storage vinculada ao nosso app
const storage = getStorage(app);

export async function fazerUploadDeArquivo(arquivo, caminhoPasta =
"imagens") {
    try {
        // 1. Cria uma referência única para o arquivo usando o
nome original dele
        const nomeArquivoUnico = `${Date.now()}_${arquivo.name}`;
        const storageRef = ref(storage,
`caminhoPasta/{nomeArquivoUnico}`);

        console.log("Iniciando o upload do arquivo...");

        // 2. Faz o upload dos bytes brutos do arquivo para o
Storage
        const snapshot = await uploadBytes(storageRef, arquivo);
        console.log("Arquivo enviado com sucesso para a nuvem!");

        // 3. Obtém a URL pública de download para exibir a imagem
ou salvar no Firestore
        const urlDownload = await getDownloadURL(snapshot.ref);
        console.log("URL pública do arquivo:", urlDownload);

        return urlDownload;
    } catch (error) {
        console.error("Erro ao realizar o upload:",
error.message);
        throw error;
    }
}
```

5.4. Integrando com o Frontend (Exemplo com Input de Arquivo em React)

Para permitir que o usuário selecione um arquivo direto do computador ou celular, utilizamos um elemento HTML do tipo <input type="file" /> acoplado ao nosso serviço de upload.

```
import { useState } from 'react';
import { fazerUploadDeArquivo } from './storage';

function TelaUpload() {
  const [arquivoSelecionado, setArquivoSelecionado] =
useState(null);
  const [urlImagem, setUrlImagem] = useState('');
  const [enviando, setEnviando] = useState(false);

  // Captura o arquivo escolhido pelo usuário no input
  function handleArquivoChange(e) {
    if (e.target.files[0]) {
      setArquivoSelecionado(e.target.files[0]);
    }
  }

  async function handleUploadClick() {
    if (!arquivoSelecionado) {
      alert('Selecione um arquivo primeiro!');
      return;
    }

    try {
      setEnviando(true);
      // Chama nossa função de serviço do Storage
      const url = await fazerUploadDeArquivo(arquivoSelecionado,
'perfis');
      setUrlImagem(url);
      alert('Upload concluído com sucesso!');
    } catch (err) {
      alert('Falha ao enviar arquivo.');
    } finally {
      setEnviando(false);
    }
  }

  return (
    <div style={{ padding: '20px' }}>
      <h2>Upload de Imagem para o Firebase Storage</h2>

      <input type="file" accept="image/*"
onChange={handleArquivoChange} />
```

```
      <br /><br />

      <button
        onClick={handleUploadClick}
        disabled={enviando}
        style={{ padding: '8px 16px' }}
      >
        {enviando ? 'Enviando...' : 'Fazer Upload'}
      </button>

      {/* Exibe a imagem na tela usando a URL retornada pelo
Firebase */}
      {urlImagem && (
        <div style={{ marginTop: '20px' }}>
          <h4>Pré-visualização:</h4>
          <img src={urlImagem} alt="Preview" style={{ width:
'200px', borderRadius: '8px' }} />
        </div>
      )}
    </div>
  );
}

export default TelaUpload;
```